ChatGPT

BoundaryLab: The Physical AI Project I Would Bet the Hackathon On

Deep-research recommendation for the Nebius × NVIDIA Global AI Hackathon, verified against the live competition rules, current NVIDIA/Nebius product direction, current senior engineering roles, official repositories, and recent robotics research as of October 24, 2026.

PDF: Download the complete BoundaryLab Physical AI deep-research report

The conclusion first

After looking beyond generic hackathon brainstorming and asking a different question — **"What difficult Physical AI problem are NVIDIA and Nebius themselves signaling that they care about?"** — I would not build another factory dashboard, ordinary digital twin, robot copilot, autonomous pick-and-place demo, or "LLM controls a robot" application.

I would build:

> **BoundaryLab — CI/CD for Physical AI**
>
> **The crash-test lab for robot intelligence.**
>
> BoundaryLab automatically finds the *smallest change in the physical world that breaks a robot policy*, turns that failure into a reproducible regression test and targeted training data, and blocks deployment until the next policy version survives that edge case.

In five-year-old language:

Car companies crash-test cars before selling them. BoundaryLab crash-tests robot brains before allowing them to ship.

A robot appears to work perfectly.

BoundaryLab starts changing the world around it:

```
friction
camera angle
object location
object weight
lighting
```

```
sensor noise
network / perception delay
actuator strength
occlusion
unexpected obstacles
```

Eventually the robot fails.

But BoundaryLab doesn't simply say:

> Robot failed.

It asks:

> **What is the smallest possible change that made it fail?**

Maybe the robot works with:

```
Camera offset:     2°
Sensor delay:     20 ms
Object friction:  normal
```

but fails when the combination reaches something like:

```
Camera offset:     +6.8°
Sensor delay:      73 ms
Surface friction:  -11%
```

Those numbers would come from the actual experiment; you would **never pre-script them**.

BoundaryLab then saves that exact world, seed, simulator version, policy version, trajectory and video. That becomes a permanent regression test.

Then the system can create targeted synthetic examples around the failure region, fine-tune or augment the robot policy, and run the complete suite again.

The final output is brutally simple:

```
POLICY RELEASE CANDIDATE
GR00T-policy-v17

Nominal benchmark:                PASS
Random robustness benchmark:      PASS
```

```
BoundaryLab adversarial search:     FAIL

Minimal counterexample discovered:
camera yaw + sensor latency + low friction

Deployment decision:
                      ❌ NO SHIP
```

After repair:

```
POLICY RELEASE CANDIDATE
GR00T-policy-v18

Previous counterexample:            PASS
Held-out adversarial cases:         PASS
Regression suite:                   PASS

Deployment decision:
                      ✅ SHIP
```

That is substantially different from the majority of AI-hackathon projects because **the product is not the robot. The product is the engineering system that decides whether the robot intelligence is trustworthy enough to release.**

That direction maps unusually well to what NVIDIA and Nebius are publicly building and hiring for right now. NVIDIA's current senior/principal/director postings repeatedly emphasize high-fidelity simulation, world models, long-tail scenarios, deterministic real-time systems, observability, fallback behavior, evaluation harnesses and catching failures before production. Nebius currently lists Physical AI infrastructure roles and has a new Physical AI Workbench explicitly covering simulation, robot-policy training, synthetic data and evaluation. That alignment is an inference from their public materials, not a claim about an unpublished internal roadmap. [1]

And one correction to the competitive picture: the live Devpost page I checked now shows **26,937 registered participants**, rather than 30,000. Registration is not the same thing as a completed qualifying submission, so the final judging pool should not be assumed to contain 26,937 finished projects. [2]

What the sponsors' senior engineering roles reveal

I deliberately looked at the problem from the perspective of the people who might be judging or evaluating the engineering quality rather than from the perspective of “what app can we make with an API?”

The signal is surprisingly consistent.

NVIDIA is trying to bridge learned intelligence and engineering assurance

One of the strongest findings was NVIDIA's current **Principal Engineer, Autonomous Vehicles and Physical AI Solutions** role. NVIDIA describes responsibilities spanning Cosmos world foundation models, synthetic world generation, complex long-tail scenarios, safety architecture, deterministic real-time design, observability and high-fidelity simulation. Particularly relevant to your project is its emphasis on engineering methodologies that identify problems before production. [3]

That is basically the problem BoundaryLab attacks.

NVIDIA's current Director of Engineering role for autonomous vehicles is even more revealing. NVIDIA explicitly says the next generation of autonomy should not be purely classical robotics or purely end-to-end neural networks; it describes an architecture integrating classical safety-focused systems with learned driving systems, foundation models and world models. The responsibilities include redundancy, fallback, degraded-mode operation, VLA models and system-level safety. [4]

That means a project architecture like:

```
        Learned intelligence
         GR00T / Nemotron
               |
               v
     candidate action / policy
               |
               v
   +--------------------------+
   | deterministic test layer |
   | constraints              |
   | regression tests         |
   | temporal requirements    |
   +--------------------------+
               |
         PASS / FAIL
```

is much closer to NVIDIA's publicly stated engineering philosophy than:

```
LLM
 |
 v
robot
```

There is another signal at the systems level. NVIDIA's Senior Software Engineer role for **Low Latency Physical AI Sensor Systems** centers on getting sensor data to GPU memory through technologies such as RDMA and GPUDirect and operating under strict latency and throughput constraints. [5]

That gives BoundaryLab an especially interesting adversarial dimension:

> **Timing itself becomes physics.**

Most teams will vary objects and camera positions.

You can vary:

$$\text{observation latency}$$

$$\text{jitter}$$

$$\text{frame loss}$$

$$\text{sensor synchronization error}$$

because a fantastic policy receiving stale information is still a bad physical system.

For example:

```
World hasn't changed.

Robot hasn't changed.

Object hasn't changed.

Only perception latency changed:
20ms → 35ms → 50ms → 80ms

At what point does policy behavior become unsafe?
```

That begins to look like serious cyber-physical systems engineering.

NVIDIA's Director of Product Management for Agent Tooling similarly calls out agent harnesses, evaluation systems, benchmarking, observability, workflow completion and production readiness across AI, robotics, simulation and HPC. [6]

That suggests another way to frame BoundaryLab:

> **It is an evaluation harness for embodied agents.**

Not just an application built *with* an agent.

Nebius is building exactly the infrastructure you need

Nebius's careers site currently lists **Forward Deployed Engineer, Physical AI Infrastructure** roles, alongside GPU architecture, Token Factory and efficient-inference positions. That strongly suggests Physical AI infrastructure is not a side experiment for the company. [7]

More importantly, Nebius has now released a public **Physical AI Workbench**.

Its official repository describes the system as an agent-facing control plane for:

```
infrastructure
simulation
robot policy training
synthetic data
evaluation
```

on Nebius infrastructure. Its `npa` tooling can orchestrate containerized workloads, manage durable artifacts, run workflow graphs and integrate technologies including Isaac Lab, Genesis, LeRobot, GR00T and Cosmos. The repository also describes a canonical Sim2Real pipeline and emphasizes validation, provenance and the difference between merely running a workload and proving policy performance. [8]

This is extremely timely.

Instead of building a generic LangGraph agent around AWS or your laptop and adding one Nebius API call at the end, BoundaryLab could use the sponsor's own emerging Physical AI control plane as a major architectural layer:

```
BoundaryLab
     |
     v
npa workflow
     |
     +---- Isaac Lab
     |
     +---- Genesis
     |
     +---- GR00T
     |
     +---- Cosmos
     |
     +---- evaluation
     |
     +---- artifacts
     |
```

```
    v
Nebius AI Cloud GPUs
```

Nebius's own Physical AI Living Lab also combines NVIDIA Cosmos, Isaac Sim/Lab, the Physical AI Data Factory and Nebius GPU infrastructure to move teams through physical-AI development workflows. [9]

So my synthesis of the company signals is:

Frontier	NVIDIA signal	Nebius signal	BoundaryLab
World models	Cosmos	Cosmos workflows	Adversarial worlds
VLA policies	GR00T	GR00T Workbench support	Policy under test
Simulation	Isaac	NPA / Isaac / Genesis	Test laboratory
Long-tail failures	Explicit priority	Synthetic-data pipelines	Core problem
Evaluation	Senior tooling roles	Workbench evaluation	Core product
Observability	Principal roles	Artifact workflows	Replay + provenance
Safety / fallback	Director-level priority	Pipeline gates	Release gate
Scale	GPU computing	Nebius GPU cloud	Parallel rollouts
Agentic workflows	NVIDIA tooling	NPA agent control plane	Test orchestration

I did **not** find a relevant verifiable current VP-level Physical AI job posting in the sources I reviewed, so I am deliberately not inventing one. The principal and director postings provide plenty of evidence for the engineering direction.

The research insight that makes BoundaryLab technically serious

The idea becomes significantly stronger when you stop thinking about it as “generate difficult simulations” and formulate it as a **falsification problem**.

Suppose your robot succeeds in its standard environment.

Your nominal world is:

$$x_0$$

where x represents the physical world:

$$x = [\text{friction}, \text{mass}, \text{pose}, \text{camera offset}, \text{latency}, \text{noise}, \text{lighting}, \dots]$$

Your robot policy is:

$$\pi$$

The robot generates a trajectory:

$$\tau(\pi, x)$$

and you define an actual engineering requirement:

$$\phi$$

For example:

> pick up the object within 12 seconds while never colliding with the table and never exceeding a force limit.

Define:

$$\rho(\tau, \phi)$$

as how strongly that trajectory satisfies the requirement.

Positive:

$$\rho > 0$$

means the test passes.

Negative:

$$\rho < 0$$

means it violates the requirement.

BoundaryLab asks:

$$x^* = \arg\min_x D(x, x_0)$$

subject to:

$$\rho(\tau(\pi, x), \phi) < 0$$

In ordinary English:

> **Find the world closest to normal that breaks the robot.**

That is the project.

Why this is a real research problem

A 2026 *Journal of Machine Learning Research* paper, **Causal Falsification of Digital Twins**, argues that observational data generally cannot certify that a digital twin is causally correct without strong assumptions. The authors instead develop methods for finding situations in which a twin can be shown to be wrong. [10]

That gives you a powerful philosophical foundation:

> Don't try to prove your simulator or robot is universally correct.
>
> **Try extremely hard to prove it wrong.**

There is also already 2026 research proposing adversarially generated synthetic robot-safety scenarios using a Red Team that constructs hazards and a Blue Team that improves policies. That means you should **not** claim you invented adversarial robot testing. [11]

Your novelty should instead be the integrated engineering loop:

```
FIND FAILURE
     ↓
MINIMIZE FAILURE
     ↓
EXPLAIN FAILURE
     ↓
STORE AS REGRESSION TEST
     ↓
GENERATE TARGETED DATA
     ↓
REPAIR POLICY
     ↓
RE-RUN ALL PREVIOUS FAILURES
     ↓
RELEASE / BLOCK
```

That makes it **Physical AI CI/CD**.

Recent robotics research strengthens this thesis from another direction. NVIDIA-affiliated RoboTTT research demonstrates test-time-training robot policies with extremely long visuomotor context and reports increased robustness and on-the-fly policy improvement; separate 2026 work on Q-planning explores learning from failed deployment rollouts without retraining the entire base policy. [12]

BoundaryLab therefore provides a natural upstream component:

> Where do the valuable failures for self-improvement come from?

Your system **hunts them**.

The Failure Atlas

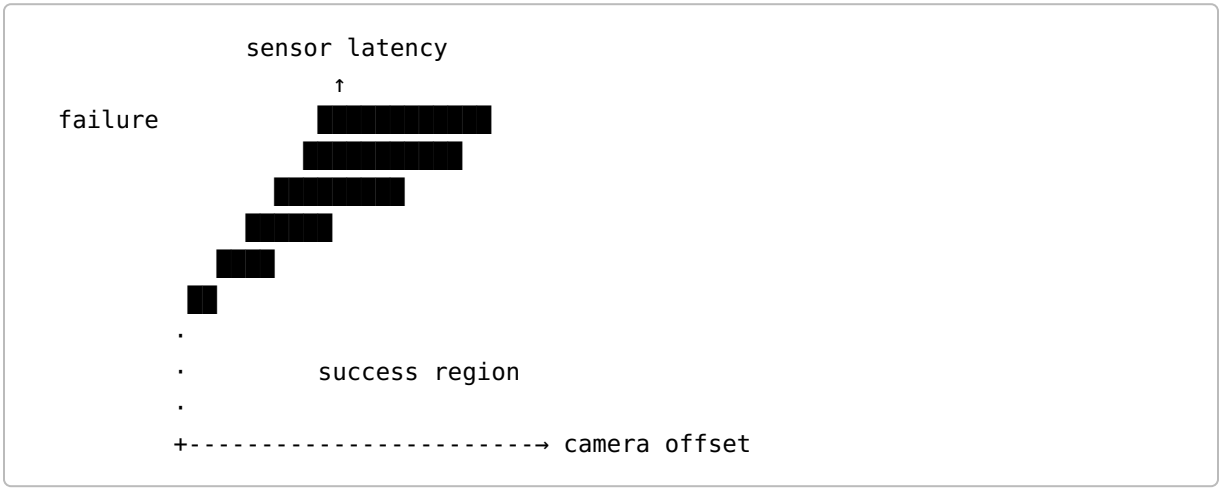
This can become your visually memorable technical innovation.

Instead of one success score:

```
GR00T success rate: 88%
```

show a **failure surface**.

Imagine two parameters first:



Then let the user select:

```
friction
payload
camera shift
latency
lighting
noise
```

The important quantity becomes:

> **How far is the robot from failure?**

You can define something conceptually similar to a robustness margin:

$$M(\pi) = \min_{x:\text{failure}} D(x, x_0)$$

A large margin means the policy has to encounter a substantially different world before breaking.

A tiny margin means the nominal benchmark may be giving a false sense of security.

That feels much more frontier-level than another “88% accuracy” gauge.

Counterexample minimization is your money feature

Suppose your adversarial search finds:

```
low friction
camera +9°
100ms delay
heavy object
dark lighting
partial occlusion
```

and the robot drops the object.

That's interesting but messy.

Now BoundaryLab starts removing conditions:

```
remove darkness       → still fails
remove occlusion      → still fails
normal object mass    → still fails
increase friction     → passes
reduce camera offset  → passes
reduce latency        → passes
```

It eventually discovers:

```
Minimal reproducible failure:

camera yaw = +6.7°
perception delay = 74 ms
friction coefficient = 0.39
```

Now you haven't merely generated an adversarial scene.

You have discovered a **failure mechanism**.

That is the part I expect experienced engineers to remember.

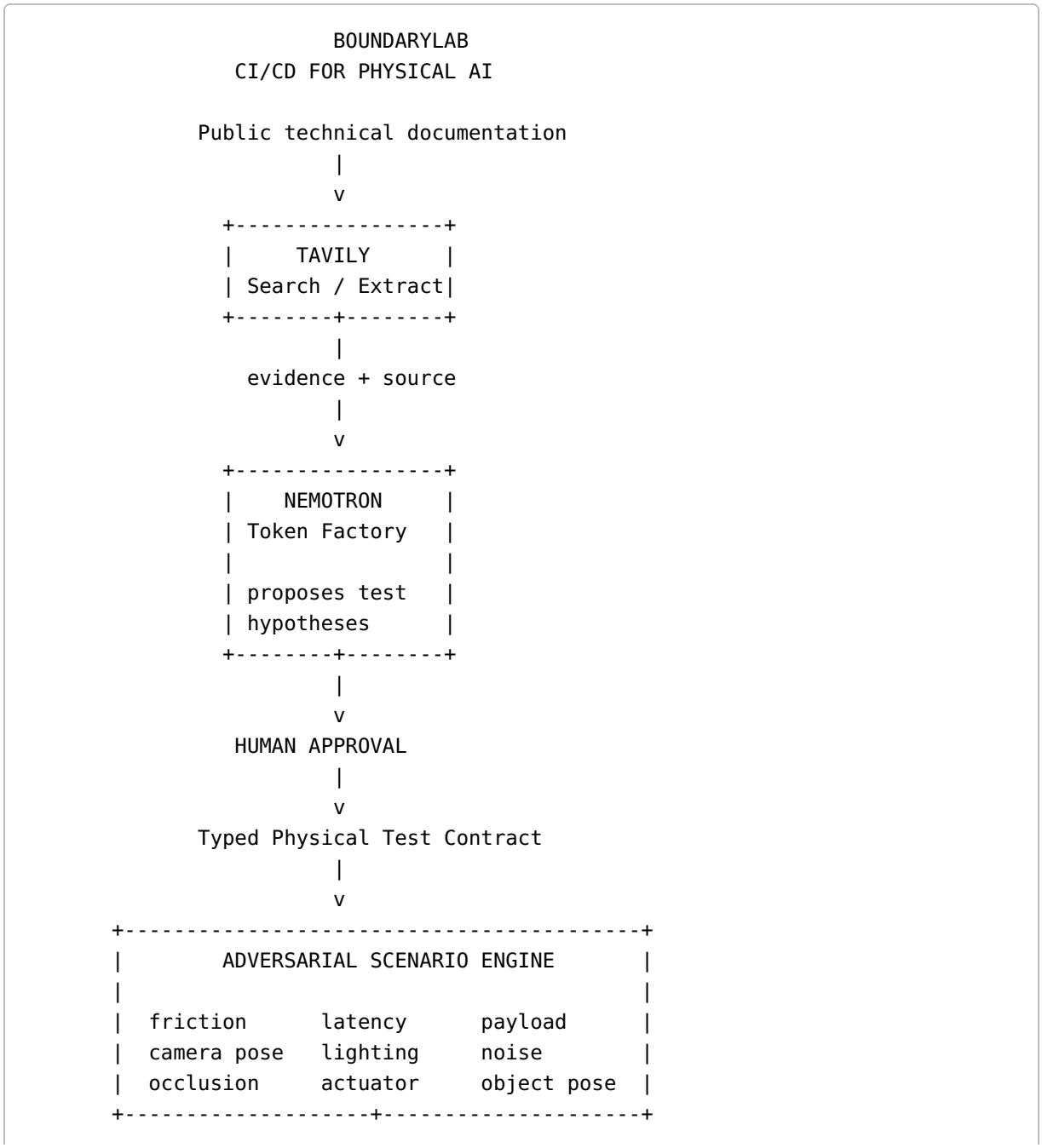
The NVIDIA + Nebius + Tavily architecture

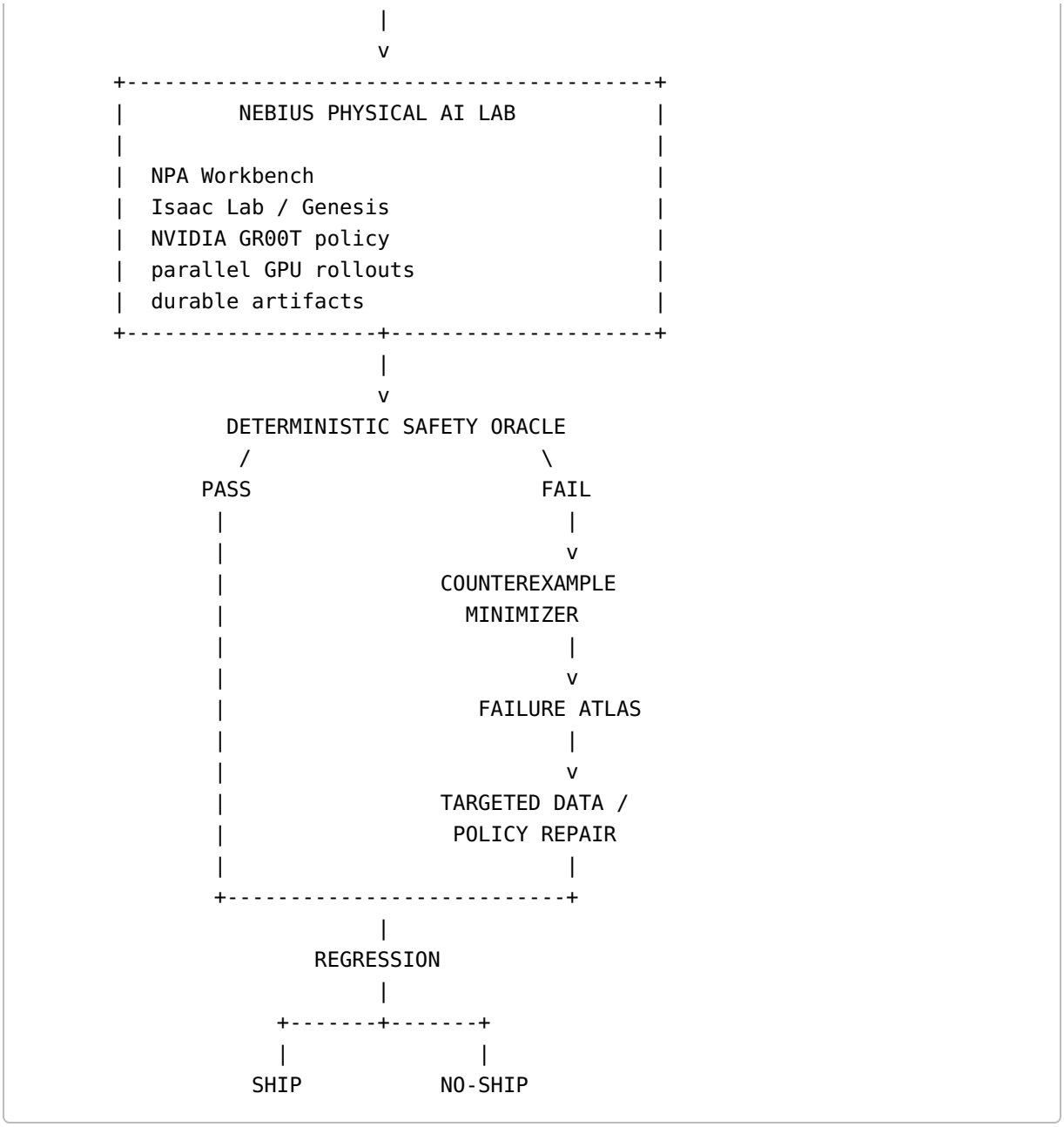
The project should be architected around a very important principle:

> **Generative AI proposes. Deterministic engineering decides.**

Do **not** let an LLM decide that a robot is safe.

The architecture should look like this:





Nemotron

Nemotron's role should be **scientific test reasoning**, not robot motor control.

Give it:

```
{
  "task": "pick object and place in bin",
  "nominal_environment": {...},
  "documented_constraints": {...},
```

```
  "previous_failures": [...],
  "available_variables": [...]
}
```

and require structured output such as:

```
{
  "hypothesis": "visual-to-action delay may interact with camera yaw",
  "variables": [
    {
      "name": "camera_yaw_deg",
      "min": -10,
      "max": 10
    },
    {
      "name": "observation_delay_ms",
      "min": 0,
      "max": 120
    }
  ],
  "reason_to_test": "...",
  "expected_failure_signature": "..."
}
```

Pydantic validates that structure.

Nemotron cannot return:

```
{"safe": true}
```

and have that become authoritative.

Instead:

```
Nemotron
   |
   v
hypothesis

Physics simulation
+
deterministic requirements
   |
```

```
   v
evidence
```

That separation is the same broad learned-system/classical-safety philosophy NVIDIA describes in current senior autonomy hiring. [4]

GR00T N1.7

The ideal stretch target is for **GR00T N1.7 to be the actual policy BoundaryLab attacks**.

NVIDIA's current official GR00T repository describes N1.7 as an open vision-language-action model for generalized robot skills, with a Cosmos-Reason2-2B VLM backbone, a workflow for fine-tuning/evaluation/deployment, and ONNX/TensorRT support. It also reports pretraining that includes 20,000 hours of EgoScale human video. [13]

The pipeline then becomes:

```
GR00T N1.7
      |
      v
robot policy
      |
      v
BoundaryLab
      |
      +--- find weaknesses
      |
      +--- generate hard cases
      |
      +--- optionally fine-tune
      |
      +--- regression test
```

But **do not allow GR00T integration to sink the project**.

Your priority order is:

```
1. Working adversarial test loop
2. Real Nebius execution
3. Reproducible counterexample
4. Nemotron integration
5. Evaluation
```

```
6. GR00T as policy under test
7. Cosmos
```

If GR00T's embodiment integration becomes a two-day rabbit hole, use a smaller robot policy initially while retaining the required NVIDIA integration through Nemotron.

Cosmos 3

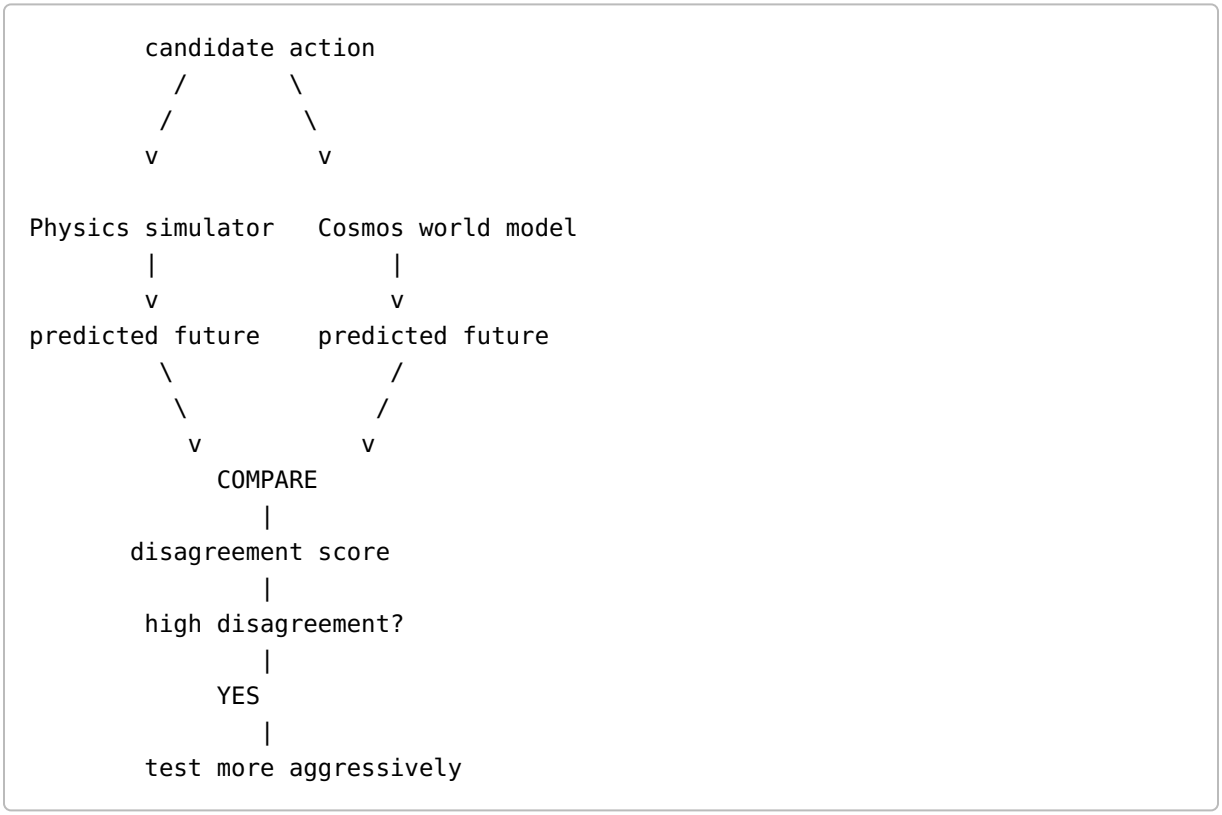
Cosmos belongs in the **stretch layer**, not the critical path.

The official Cosmos 3 repository describes an open Physical AI world-model platform that supports inference stacks including vLLM, vLLM-Omni, SGLang, TensorRT-LLM and NVIDIA NIM. NVIDIA also explicitly documents potential temporal inconsistency, object morphing and physically implausible dynamics and says safety-critical use requires additional system-level validation. [14]

That limitation creates a fascinating BoundaryLab feature:

World-model disagreement

Instead of trusting one simulator:



In plain English:

When two models of reality disagree, that's exactly where we should investigate.

That is a beautiful research-extension story.

But it is not necessary for your core hackathon result.

Nebius Physical AI Workbench

This may be one of your strongest competitive advantages because it is extraordinarily current.

The official repository says Workbench is the Physical AI control plane on Nebius and exposes workflows for simulation, robot-policy training, generation, evaluation and artifact inspection. It explicitly lists lifecycle-bearing tools for LeRobot, Cosmos, GR00T, Genesis and Isaac Lab. [15]

Instead of:

```
LangChain → API → dashboard
```

you can show:

```
BoundaryLab
     |
npa.workflow
     |
     +── generate scenario
     +── run simulator
     +── evaluate trajectory
     +── store artifact
     +── update failure search
     +── repeat
```

NPA's current workflow documentation describes declarative state-machine specs with states, transitions and artifact contracts. [16]

That is exactly the sort of deterministic orchestration I would favor over a free-running swarm of agents.

You can still use LangGraph for a research/explanation agent.

But:

```
LLM orchestration         Physical testing
------------------        ------------------------
LangGraph optional        NPA workflow
```

```
flexible                  deterministic
reasoning                 state transitions
```

This is a more mature architecture.

Nebius Serverless Jobs and Endpoints

The hackathon's Physical AI description explicitly suggests Serverless Jobs for simulation, synthetic-data generation, robot-policy evaluation and sensor processing, and Serverless Endpoints for real-time inference. [2]

So if Serverless Jobs access works in your account, a highly sponsor-visible deployment would be:

```
BoundaryLab API
      |
      +-------------------------+
      |                         |
      v                         v
Serverless Job            Token Factory
batch adversarial         Nemotron reasoning
simulation
      |
      v
100–1000 rollouts
```

However, I would **not invent a Serverless Jobs API invocation until your team verifies the live documentation and account access**.

Nebius's public NPA repository gives you a current, verified alternative orchestration path through Workbench/SkyPilot. [17]

Tavily

Tavily can become far more interesting than:

> “The robot agent searches the Internet.”

Tavily's current product exposes web search and extraction capabilities designed for agent workflows. [18]

Make it the **engineering evidence layer**.

Suppose the user selects a robot model.

BoundaryLab asks Tavily for public manufacturer documentation.

Tavily retrieves the source.

Nemotron proposes:

```
{
  "property": "maximum_payload",
  "value": 3.0,
  "unit": "kg",
  "source": "...",
  "confidence": 0.94
}
```

Then:

```
Tavily
   |
   v
source evidence
   |
   v
Nemotron extraction
   |
   v
HUMAN CONFIRMATION
   |
   v
approved test constraint
```

The crucial safety boundary:

> **A webpage can never directly change the robot's safety limits.**

Tavily provides evidence.

Nemotron interprets.

A human approves.

The deterministic test engine enforces.

That is a sophisticated use of RAG/search.

And the Best Use of Tavily award is a $3,000 bonus award requiring a functional runtime Tavily API call. [19]

What you should actually build before the deadline

You do **not** have eight days anymore.

Today is **October 24, 2026**. The official deadline is **Friday, October 30, 2026 at 10:00 AM Pacific / 1:00 PM Eastern**. [20]

That means your real development deadline should be **October 29**.

The strategy therefore has to be ruthless.

The core vertical slice

Use one robot.

One task.

Four or five adversarial variables.

One policy.

One extremely impressive end-to-end loop.

I would pick a tabletop manipulation problem because judges can understand it instantly:

```
Robot arm
   ↓
pick object
   ↓
transport object
   ↓
place into target
```

Now parameterize:

```
object position
surface friction
object mass
camera yaw
observation latency
```

Your core experiment is:

Tester	Budget	Question
Nominal benchmark	fixed	Does it work under normal conditions?
Random domain randomization	200 runs	Can random testing discover failures?
BoundaryLab	200 runs	Can targeted active search discover failures faster and closer to nominal?

That comparison is extraordinarily important.

Otherwise a judge can reasonably ask:

> Why do I need your fancy AI system? Why not randomly change simulator settings?

You need data.

For example, after running the experiment you might discover:

```
                     Episodes before first failure
Nominal benchmark                 none
Random search                      87
BoundaryLab                        21
```

Those are **illustrative**, not numbers to put into your submission.

Your real results determine the claim.

What the dashboard shows

I would design only four views.

Live Test Lab

```
Policy: groot-release-17

GPU workers: 8
Experiments: 142 / 250
Failures: 17
Nearest counterexample distance: 0.083

[ LIVE SIMULATION ]
```

Failure Atlas

```
Success probability
         ^
         |
      safe
         |
---------boundary-------------
         |
      failure
         +-------------------->
           sensor latency
```

Counterexample

```
Policy normally succeeds.

BoundaryLab discovered:

camera_yaw:       +6.7°
latency:          74ms
friction:         0.39

Result:
OBJECT DROPPED

Replay seed:
782314
```

Release Gate

```
POLICY 17

Nominal tests          ✅
Previous regressions   ✅
Adversarial boundary   ❌

                DO NOT SHIP
```

That is enough.

The six-day build order

Date	What must exist by the end of the day
Oct 24	Token Factory call + NPA install + one real Nebius simulator run + public repo

Date	What must exist by the end of the day
Oct 25	Robot policy + scenario schema + deterministic success/failure evaluator
Oct 26	100+ reproducible runs + random baseline + failure replay
Oct 27	Active adversarial search + counterexample minimization + Failure Atlas
Oct 28	Tavily evidence + repair/recovery experiment + full regression benchmark
Oct 29	Video, README, plots, source map, feedback, submission
Oct 30	Emergency buffer only

The hackathon's own guidance recommends submitting early rather than relying on last-minute video uploading or debugging. [21]

The scope priority should be:

```
MUST HAVE

1. real Nebius workload
2. live simulator
3. policy under test
4. adversarial search
5. deterministic failure test
6. minimal counterexample
7. reproducible replay
8. measured comparison against random testing
9. real Nemotron call
10. real Tavily call
```

Then:

```
STRETCH

11. GR00T N1.7 policy under test
12. targeted fine-tuning / repair
13. Cosmos visual augmentation
14. cross-model disagreement
```

And only then:

```
MOONSHOT

15. edge runtime guard
```

```
16. real robot / Jetson
17. multiple robot embodiments
18. multi-simulator validation
```

Do **not** build five production lines.

Do **not** build five robots.

Do **not** build a fancy factory.

Build one experiment that makes a principal robotics engineer say:

> Wait. Run that again.

How this becomes a Grand Prize submission rather than only a research demo

The official competition has a viability/fit gate and then scores qualifying projects on four equally weighted dimensions: **Technological Implementation, Design, Potential Impact and Quality of the Idea**. The organizers explicitly say a scripted Physical AI demo is table stakes and encourage teams to push beyond it. [22]

So BoundaryLab needs to win all four dimensions, not only the technical one.

Judging criterion	What you show
Technological Implementation	Real Nebius GPU simulations, NVIDIA models, active search, reproducibility, policy evaluation
Design	A coherent test → failure → repair → release product, not a collection of notebooks
Potential Impact	A concrete product for robotics ML / validation engineers trying to release learned policies
Quality of Idea	“CI/CD for robots” and minimal failure-boundary discovery rather than another robot-agent wrapper

The three-minute video

The competition requires a public YouTube video no longer than three minutes. A Physical AI project without hardware may use simulation, but at least one minute must visibly show the important application modules operating. [23]

I would make the video approximately:

0:00–0:15

Robot succeeds.

Voice:

> "This policy passes its benchmark. Would you ship it?"

0:15–0:35

BoundaryLab starts.

```
Searching 8 dimensions
GPU rollouts on Nebius
```

0:35–1:00

Failure appears.

Robot drops object.

Voice:

> "BoundaryLab found a world the benchmark didn't."

1:00–1:25

Watch the minimizer remove unnecessary perturbations.

```
lighting       ❌ irrelevant
object mass    ❌ irrelevant
occlusion      ❌ irrelevant

camera yaw     ✅
latency        ✅
friction       ✅
```

Then:

> "This is the smallest world change we found that breaks the policy."

That is your **signature scene**.

1:25–1:45

Show Tavily source → Nemotron structured hypothesis → approved test contract.

1:45–2:10

Targeted repair or recovery.

Run the exact counterexample again.

2:10–2:35

Run all previous counterexamples.

Show:

```
release-17    FAIL
release-18    PASS
```

2:35–2:50

Show architecture quickly:

```
NVIDIA GR00T / Cosmos
        +
Nebius Physical AI Workbench
        +
Nemotron Token Factory
        +
Tavily evidence
```

2:50–3:00

End on:

> **“BoundaryLab. Crash-test robot intelligence before reality does.”**

That is a much stronger ending than explaining what a VLA is.

Your measurable research table

This should appear in the README.

No fake numbers.

Evaluation	Nominal	Random testing	BoundaryLab
Episodes used	actual	actual	actual
Failures found	actual	actual	actual
Episodes to first failure	actual	actual	actual
Closest failure distance	actual	actual	actual
Reproducible failures	actual	actual	actual
GPU time	actual	actual	actual

Then, if you get the repair loop working:

Metric	Before	After repair
Original counterexample	fail/pass	fail/pass
Held-out adversarial success	measured	measured
Regression suite success	measured	measured
Nearest discovered failure	measured	measured

This is what prevents the project from being dismissed as an attractive demo.

One especially sophisticated addition: temporal requirements

Instead of evaluating only:

```
collision == False
```

you can define temporal constraints such as:

> The robot must grasp the object, then lift it, then enter the target region, and do so within 12 seconds without exceeding force limits.

Recent work has used Signal Temporal Logic for trajectory-level robot-policy requirements, including ordered behavior, deadlines and physical interaction constraints. [24]

Your simplified test DSL could look like:

```
ALWAYS force < FORCE_LIMIT

ALWAYS distance_to_human > SAFE_DISTANCE
```

```
EVENTUALLY[0,5] object_grasped

EVENTUALLY[0,12] object_in_target

NEVER collision
```

Now BoundaryLab is not testing:

> Did the object approximately move?

It is testing a specification.

That's a serious engineering step.

One especially frontier extension: runtime failure-boundary awareness

After BoundaryLab has discovered thousands of simulation outcomes, train a small risk estimator:

$$R(s_t) \rightarrow P(\text{failure in next horizon})$$

Then deploy that **at the edge**, not a giant cloud model inside the motor-control loop.

```
Sensors
   |
   v
fast local risk model
   |
   +------ normal ------> learned policy
   |
   +------ risky -------> fallback / stop / human
```

This strongly aligns with NVIDIA's public emphasis on real-time Physical AI systems, sensor-to-GPU infrastructure, deterministic fallback and edge deployment. [25]

You can explain it in one sentence:

> “The cloud discovers failure boundaries; the edge learns to recognize when the robot is approaching one.”

That is a fantastic future-work statement.

The rules, grounding discipline, and the final competitive verdict

BoundaryLab also fits the actual competition rules.

The hackathon requires a submission to run on Nebius Token Factory or Nebius AI Cloud and use at least one NVIDIA open model. The Physical AI description specifically names embodied/edge agents, robotics, IoT and on-device intelligence and encourages the use of Nebius infrastructure for simulation, synthetic data and policy evaluation. Simulation-only projects are explicitly permitted by the current FAQ. [23]

The official deadline is **October 30, 2026 at 10:00 AM Pacific**, with judging in December and winners expected around January 11, 2027. The Grand Prize is **$20,000**, the Physical AI track prize is an NVIDIA Jetson Orin Nano, and the Tavily bonus is **$3,000**; the rules state that a project can receive an overall award *or* a track award plus one bonus award, rather than stacking overall prizes. [19]

Most importantly, implement the grounding discipline you requested.

Rule	What BoundaryLab does
Source first	No API/model/command enters README before verification in current official docs
Real sponsor calls	Preserve real Nebius run IDs, Token Factory calls and Tavily calls
Typed AI outputs	Pydantic/JSON schema between Nemotron and the test system
Human approval	Internet-derived constraints never silently become executable
Deterministic safety	LLM cannot override a failed physical constraint
Reproducibility	Policy + simulator + seed + config + artifacts versioned together
No fake metrics	Video numbers generated from benchmark artifacts
No simulator overclaim	Say “robust within tested simulation envelope,” never “certified safe”
No kitchen-sink stack	Every tool must have a load-bearing reason to exist

There is a particularly important honesty point about novelty.

I **cannot substantiate a claim that “nobody has thought of this.”** Research already exists on adversarial synthetic robot-safety scenarios, digital-twin falsification, active adaptation and failure-driven policy improvement. [26]

That's actually good news.

It means you're working on a **real research frontier**, not an invented hackathon problem.

Your novelty claim should be narrower and more credible:

> **BoundaryLab turns adversarial Physical AI testing into a reproducible, counterexample-minimizing CI/CD system built natively across NVIDIA robot/world models and Nebius Physical AI infrastructure.**

That is a claim you can prove with the project.

And I would rank the concepts we have discussed like this:

Concept	Research depth	Sponsor alignment	Visual impact	Six-day deliverability	My choice
Factory digital twin	High	High	High	High	Good
WorldForge recovery robot	High	High	Very high	Medium	Strong
RealityBreak self-correcting physics	Very high	Very high	High	Medium-low	Excellent research
BoundaryLab adversarial Physical AI CI	**Very high**	**Exceptional**	**Very high**	**Medium-high**	**Build this**

RealityBreak asked:

> Can an AI realize its model of reality is wrong?

BoundaryLab takes that idea and makes it a more complete **product and infrastructure thesis**:

> **Before a Physical AI policy ships, can an autonomous system actively discover where it breaks, explain the smallest counterexample, repair it, and prove that the fix didn't create new failures?**

That is closer to the intersection of the problems the current NVIDIA principal/director roles describe and the workflows Nebius is actively exposing through Physical AI Workbench. [27]

The part I would protect even if everything else has to be cut is **minimal counterexample discovery**.

The unforgettable demo is not:

> “Look, our robot works.”

It is:

> **“Look, your robot looked like it worked. BoundaryLab found the tiny world change that proves it doesn't.”**

Then you press replay.

The robot fails again.

Same seed.

Same conditions.

Same result.

Then you repair the policy.

Replay.

It succeeds.

Run every previous counterexample.

All green.

That's the Grand Prize story.

[Download the full source-mapped BoundaryLab deep-research PDF]

---

1 3 27 Principal Engineer, Autonomous Vehicles and Physical AI Solutions
https://nvidia.wd5.myworkdayjobs.com/en-US/NVIDIAExternalCareerSite/job/Principal-Engineer--Autonomous-Vehicles-and-Physical-AI-Solutions_JR2015317?utm_source=chatgpt.com

2 23 Nebius x NVIDIA Global AI Hackathon: Build the next frontier of AI on open infrastructure - Devpost
https://nebiusglobalaihackathon.devpost.com/

4 Director, Engineering - Autonomous Vehicles
https://nvidia.wd5.myworkdayjobs.com/NVIDIAExternalCareerSite/job/US-CA-Santa-Clara/Director--Engineering---Autonomous-Vehicles_JR2023529?utm_source=chatgpt.com

5 25 Senior Software Engineer, Low Latency Physical AI Sensor Systems
https://nvidia.wd5.myworkdayjobs.com/en-US/NVIDIAExternalCareerSite/job/Senior-Software-Engineer--Low-Latency-Physical-AI-Sensor-Systems_JR2020665?utm_source=chatgpt.com

6 Director, Product Management - Agent Tooling
https://nvidia.wd5.myworkdayjobs.com/en-US/NVIDIAExternalCareerSite/job/Director--Product-Management---Agent-Tooling_JR2018084?utm_source=chatgpt.com

7 Find your role: Open positions at Nebius
https://careers.nebius.com/?utm_source=chatgpt.com

8 15 17 GitHub - nebius/nebius-physical-ai · GitHub
https://github.com/nebius/nebius-physical-ai?utm_source=chatgpt.com

9 Nebius launches Physical AI Living Lab for UK and European robotics startups built with NVIDIA technologies
https://nebius.com/newsroom/nebius-launches-physical-ai-living-lab-for-uk-and-european-robotics-startups-built-with-nvidia-technologies?utm_source=chatgpt.com

10 Causal Falsification of Digital Twins
https://www.jmlr.org/beta/papers/v27/24-1752.html?utm_source=chatgpt.com

11 26 Learning of Robot Safety Policies via Adversarial Synthetic Scenarios
https://arxiv.org/abs/2606.05952?utm_source=chatgpt.com

12 RoboTTT: Context Scaling for Robot Policies
https://arxiv.org/abs/2607.15275?utm_source=chatgpt.com

13 GitHub - NVIDIA/Isaac-GR00T: NVIDIA Isaac GR00T N1.7 - A Foundation Model for Generalist Robots. · GitHub
https://github.com/NVIDIA/Isaac-GR00T?utm_source=chatgpt.com

14 GitHub - NVIDIA/cosmos: NVIDIA Cosmos is an open platform of world models, datasets, and tools that enables developers to build Physical AI for robots, autonomous vehicles, smart infrastructure, and more. · GitHub
https://github.com/NVIDIA/cosmos?utm_source=chatgpt.com

16 nebius-physical-ai/docs/workbench/npa-workflow-guide.md at main · nebius/nebius-physical-ai · GitHub
https://github.com/nebius/nebius-physical-ai/blob/main/docs/workbench/npa-workflow-guide.md?utm_source=chatgpt.com

18 Tavily Product
https://www.tavily.com/product?utm_source=chatgpt.com

19 20 Nebius x NVIDIA Global AI Hackathon: Build the next frontier of AI on open infrastructure - Devpost
https://nebiusglobalaihackathon.devpost.com/rules?utm_source=chatgpt.com

21 Nebius x NVIDIA Global AI Hackathon: Build the next frontier of AI on open infrastructure - Devpost
https://nebiusglobalaihackathon.devpost.com/updates/46205-how-to-build-a-winning-project?utm_source=chatgpt.com

22 Nebius x NVIDIA Global AI Hackathon: Build the next frontier of AI on open infrastructure - Devpost
https://nebiusglobalaihackathon.devpost.com/updates/46204-here-s-how-judging-works?utm_source=chatgpt.com

24 Learning Robot Policies from Sparse Success Signals via STL-Guided Stein Variational Policy Gradient
https://arxiv.org/abs/2609.31606?utm_source=chatgpt.com